EcoLatino News API

System Status Report & PostgreSQL Migration Roadmap

Date: July 29, 2026 | Author: Antigravity AI | Version: 1.0.0

1. Executive Summary

This report outlines the current architectural status of the EcoLatino Main API server and details a comprehensive roadmap for migrating the persistence layer to PostgreSQL. The system is built using FastAPI and SQLAlchemy (async). Currently, a split exists between an unused, in-memory SQLite database module (app/db/database.py) and a configured but unmigrated PostgreSQL setup (app/core/database.py). Moving forward, we will implement Alembic migrations to establish a production-ready PostgreSQL relational schema and standardize the repository patterns.

2. Current API Server & Codebase Status

The API layer is highly structured with standard asynchronous layers. The status is summarized below:

- Tech Stack: FastAPI (v0.140.6) running on Python 3.14.3. SQLAlchemy (v2.0.51) used for ORM models.
- Core Models: Base, TimeStampedModel, Source, SourcePolicy, Article, Story, StoryArticle, and CrawlJob.
- API Routers (/api/v1): Endpoints configured for Articles, Sources, Stories, Crawl Jobs, and Analytics.
- DB Discrepancy: app/db/database.py uses sqlite+aiosqlite:///. app/core/database.py uses postgresql+asyncpg://.
- Dependency Injection: Services (e.g. ArticleService) use dependencies.py which pulls session from app/core/database.py.
- Alembic: Initialized commands are noted in notes.txt but no migrations or config exist in the workspace.
- Containerization: docker-compose.yml is defined but currently empty.

3. PostgreSQL Database Roadmap

To set up the Postgres database, we will execute a series of steps to containerize the database instance, configure the Alembic database migration tool, generate the schema migration scripts based on our declarative SQLAlchemy models, and run the initial migration.

Database Connection Configuration (app/core/config.py):

DATABASE_URL: postgresql+asyncpg://postgres:postgres@localhost:5432/ecolatino

Note: Connection is configured for asynchronous operations using the asyncpg driver.

EcoLatino News API - Postgres Database Setup

Step-by-Step Implementation Roadmap

4. Detailed Database Setup & Migration Steps

Step 1: Containerize PostgreSQL with Docker Compose

Populate docker-compose.yml with a postgres:17 service mapping port 5432 to the host, declaring POSTGRES_USER, POSTGRES_PASSWORD, and POSTGRES_DB as 'ecolatino'.

Step 2: Environment File Setup

Create a .env file in the root workspace containing settings (e.g. DATABASE_URL) to override the default Pydantic-settings. This keeps credentials out of source control.

Step 3: Initialize Alembic

Run: uv run alembic init --template async alembic
This creates the required Alembic environment for asynchronous migrations.

Step 4: Configure Alembic to Load SQLAlchemy Models

Edit alembic/env.py to import 'Base' from 'app.models' and set target_metadata = Base.metadata. Retrieve the active connection string from settings.DATABASE_URL instead of hardcoding it.

Step 5: Generate Initial Autogenerated Migration

Run: uv run alembic revision --autogenerate -m "Initial schema"
This compares current models against the database state and generates Python migration scripts.

Step 6: Execute Migrations

Run: uv run alembic upgrade head
Applies the migrations to create tables for Source, SourcePolicy, Article, Story, and CrawlJob.

5. Immediate Next Action Items

1. Write the docker-compose.yml definition for a local PostgreSQL database server.

2. Create the root .env file and define DATABASE_URL='postgresql+asyncpg://postgres:postgres@localhost:5432/ecolatino'.

3. Remove C:\Users\v81137\Documents\EcoLatino-Main-API\app\db\database.py to resolve the code duplication.

4. Initialize Alembic, update target_metadata in env.py, and generate the initial migrations.

5. Start the Docker container, run migrations, and execute 'uv run uvicorn app.main:app --reload' to verify.